

## Questionário da Sprint 2

### 1. Por que um LLM não pode trabalhar diretamente com o texto bruto?

Uma rede neural só opera com números (multiplicação de matrizes, somas e gradientes), e uma string não tem representação numérica nem tamanho fixo. Por isso o pipeline da sprint converte o texto em tokens (3.1.py), IDs (3.2.py), vetores (3.4.py) e lotes de formato fixo (3.6.py), chegando ao tensor `[batch, contexto, embedding]` do Experimento 1, que é o formato aceito pelo modelo.

### 2. Qual é a função do vocabulário?

O vocabulário é o dicionário que liga cada token a um ID único (encode) e faz o caminho inverso (decode). Ele define o conjunto de tokens que o modelo conhece: no Experimento 3, foram 13 tokens, incluindo `<|unk|>` e `<|endoftext|>`.

Ele também protege o pipeline: no Cenário C, "medicina", "biologia" e "Java" não estavam no vocabulário e viraram `<|unk|>` (ID 11), evitando um `KeyError`. Além disso, seu tamanho define as linhas da matriz de embedding e as saídas da camada final do modelo.

### 3. Qual é a diferença entre um token e um Token ID?

O **token** é a unidade de texto legível: "engenharia", "exige", ".". O **Token ID** é o número inteiro que representa esse token, ou seja, sua posição no vocabulário.

No Cenário A do Experimento 3, os tokens `A | engenharia | exige | programação | .` viraram os IDs `[1, 7, 8, 10, 0]`. O ID não tem significado próprio: o mesmo token teria outro ID em um vocabulário construído a partir de outro texto.

### 4. Por que os Token IDs não são utilizados diretamente como representação semântica?

Porque os IDs são arbitrários. No projeto, o vocabulário é montado com `sorted(list(set(tokens_todos)))`, então os IDs seguem a ordem alfabética: "engenharia" = 7 e "exige" = 8 são vizinhos só pela letra inicial. Usados diretamente, esses números criariam relações falsas (como "programação" ser "maior" que "A"), e um único valor escalar não consegue representar as várias características de uma palavra.

### 5. Qual é a função dos embeddings?

O embedding transforma cada ID em um vetor denso de  $d$  números reais **treináveis**, usando uma tabela (`nn.Embedding`) com uma linha por token. Durante o treino, esses vetores são ajustados para que tokens usados em contextos parecidos fiquem próximos, e o significado passa a estar na geometria dos vetores, não no número do ID.

No Experimento 1, é essa etapa que transforma a matriz de IDs `[batch, contexto]` no tensor `[batch, contexto, embedding]`, como em `[2, 4] → [2, 4, 3]`.

## 6. Por que é necessário representar a posição dos tokens?

O embedding de token não depende da posição: o mesmo ID sempre gera o mesmo vetor. No Cenário B, os três pontos (ID 0) em  $[2, 0, 0, 0, 10, 11, 7, 11]$  recebem vetores idênticos. A atenção também não percebe ordem por conta própria, então "o cachorro mordeu o homem" e "o homem mordeu o cachorro" seriam a mesma entrada.

No GPT-2, a solução é um embedding posicional treinável ( $\text{contexto} \times d$ ), somado ao embedding de token sem alterar o formato do tensor:

```
x = emb_token(ids) + emb_posicao(torch.arange(tamanho_contexto))
# [B, C, d]
```

## 7. Qual é a relação entre tamanho do contexto e quantidade de amostras de treinamento?

A relação é **inversa**, mas não **inversamente proporcional**. Com stride 1, ela é **linear decrescente**:  $\text{Amostras} = N - C$ , onde  $N$  é o número de tokens do documento e  $C$  é o tamanho do contexto.

Os próprios dados do Experimento 2 ( $N = 50$ ) comprovam isso:

| Contexto (C) | Amostras | $C \times \text{Amostras}$ |
|--------------|----------|----------------------------|
| 5            | 45       | 225                        |
| 15           | 35       | 525                        |
| 30           | 20       | 600                        |
| 45           | 5        | 225                        |
| 49           | 1        | 49                         |

O produto  $C \times \text{Amostras}$  não é constante (varia de 49 a 600), então a relação não é inversamente proporcional. O que se mantém é que cada token a mais no contexto remove exatamente uma amostra.

**Validação:** no Experimento 2, as amostras bateram com  $N - C$  em todos os contextos testados. No Experimento 1, o cenário de estresse ( $C > N$ ) gerou `ValueError: __len__() should return >= 0`, confirmando o caso-limite.

A relação só se aproxima de inversamente proporcional quando o stride é igual ao contexto (janelas sem sobreposição), caso em que sobram cerca de  $N / C$  amostras.

## 8. Qual é o impacto da dimensão do embedding sobre as estruturas utilizadas pelo modelo?

A dimensão  $d$  aparece em praticamente todas as estruturas:

- **Matriz de embedding de tokens** (vocabulário  $\times$  d):  $13 \times 3 = 39$  parâmetros no projeto, e cerca de 38,6 milhões com o vocabulário do GPT-2 e  $d = 768$ .
- **Embedding posicional** (contexto  $\times$  d).
- **Tensor de entrada** [B, C, d]: a memória cresce linearmente com d.
- **Matrizes de atenção da próxima sprint** ( $W_q, W_k, W_v$ , cada uma  $d \times d$ ): os parâmetros crescem com o **quadrado** de d.

Nos cenários do Experimento 1 (float32, contando só o tensor de entrada):

| Cenário     | Shape            | Elementos   | Memória      | Atenção [B, C, C] |
|-------------|------------------|-------------|--------------|-------------------|
| Didático    | [2, 4, 3]        | 24          | 96 bytes     | 32 elementos      |
| GPT-2 Small | [4, 128, 768]    | 393.216     | 1,5 MiB      | 0,25 MiB          |
| LLM Moderno | [8, 512, 1024]   | 4.194.304   | 16 MiB       | 8 MiB             |
| Estresse    | [32, 2048, 4096] | 268.435.456 | <b>1 GiB</b> | <b>512 MiB</b>    |

Um d maior representa mais nuances de significado, mas custa mais memória, parâmetros e tempo de treino. O cenário de estresse ocuparia 1 GiB só na entrada.

## 9. Qual é a função do DataLoader no pipeline?

O **Dataset** define cada amostra (`__len__` e `__getitem__`), e o **DataLoader** agrupa essas amostras em lotes [B, C], embaralha a ordem a cada época (`shuffle`) e descarta lotes incompletos (`drop_last`).

O traceback do Experimento 1 mostra essa dependência: o DataLoader chama o `__len__` do Dataset para sortear os índices, e foi o valor negativo retornado ali que quebrou o pipeline.

Como o LLM aprende a prever o próximo token, cada amostra deve ser um par (entrada, alvo), com o alvo deslocado uma posição. Uma versão mais completa do Dataset:

```
class DatasetGPT(Dataset):
    def __init__(self, ids, tamanho_contexto, stride=1):
        if len(ids) <= tamanho_contexto:
            raise ValueError(
                f"Texto com {len(ids)} tokens < contexto {tamanho_contexto}."
            )
        self.entradas, self.alvos = [], []
        for i in range(0, len(ids) - tamanho_contexto, stride):
            self.entradas.append(torch.tensor(ids[i : i + tamanho_contexto]))
```

```

        self.alvos.append(
            torch.tensor(ids[i + 1 : i + tamanho_contexto +
1]))

    def __len__(self):
        return len(self.entradas)

    def __getitem__(self, idx):
        return self.entradas[idx], self.alvos[idx]

dataset = DatasetGPT(ids, tamanho_contexto=4, stride=1)
loader = DataLoader(dataset, batch_size=8, shuffle=True,
drop_last=True)

```

Com stride 1, essa classe gera exatamente  $N - C$  amostras.

#### 10. Quais informações produzidas nesta Sprint serão utilizadas pelo mecanismo de atenção da Sprint seguinte?

- **O tensor de embeddings**  $[B, C, d]$ , já com token + posição: é a entrada da atenção, a partir da qual são calculados Q, K e V.
- **A dimensão d**: define o  $d_{in}$  das matrizes  $W_q$ ,  $W_k$  e  $W_v$ .
- **O tamanho do contexto C**: define a matriz de atenção  $C \times C$  e a máscara causal, que impede cada token de ver os tokens futuros.
- **O tamanho do lote B**: permite processar várias sequências em paralelo.
- **O par entrada/alvo**: justifica a máscara causal; os alvos serão usados depois, no cálculo da loss.
- **O vocabulário e o token <|endoftext|>**: definem a camada de saída do modelo e as fronteiras entre documentos.